

5-Bit CARRY LOOK AHEAD ADDER

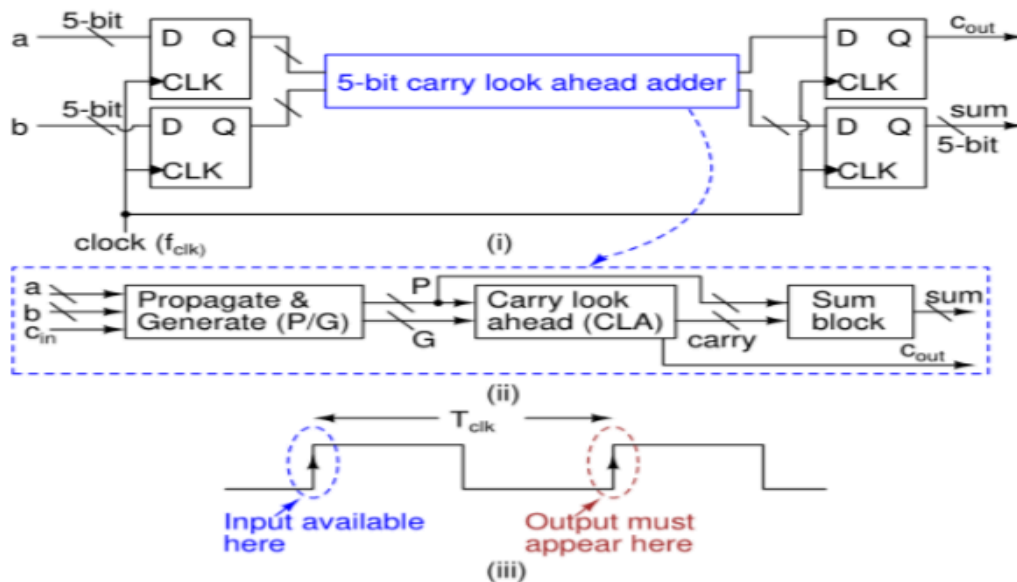
Malige Hartitha - 2024102080

Abstract—This paper presents an overview of the design, implementation, and analysis of a 5-bit Pipelined Carry Look Ahead Adder (CLA). The study focuses on the efficiency of the adder in terms of speed and resource utilization compared to conventional ripple carry adders. The implementation demonstrates reduced propagation delay using Dynamic Manchester Carry Chains and True Single Phase Clock (TSPC) logic. This work serves as a foundational step toward optimized VLSI-based arithmetic unit designs.

INTRODUCTION:

The objective of this project is to design a 5-bit Carry Look Ahead (CLA) adder using 180nm CMOS technology. To achieve superior performance and reduced area compared to static CMOS, this design utilizes a hybrid logic approach: Dynamic CMOS Logic for the critical carry propagation chain and True Single Phase Clock (TSPC) logic for the registers.

Design of a 5-Bit Carry Look Ahead(CLA):



1. Proposed structure for the Adder

Overview of the CLA Adder Architecture: The CLA adder computes carry signals in parallel, reducing delay compared to ripple-carry adders. The design consists of three main modules: the Propagate and Generate (P/G) module, the Carry Look-Ahead (CLA) module, and the Sum Block. The architecture is pipelined using TSPC D-flip-flops to synchronize inputs and outputs.

Propagate and Generate Module

The propagate (p_i) and generate (g_i) signals for each bit position ($i = 0 \text{ to } 4$) are calculated here. These signals form the basis for the CLA module.

$$p_i = a_i \oplus b_i$$

$$g_i = a_i \cdot b_i$$

Carry Look-Ahead Module

The CLA module uses p_i , g_i , and carry-in (C_{in}) values to calculate carry-out signals in parallel. The recursion formula is:

$$c_{i+1} = g_i + (p_i \cdot c_i)$$

Expanding this for the 5-bit adder:

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1(g_0 + p_0 c_0)$$

$$c_3 = g_2 + p_2(g_1 + p_1(g_0 + p_0 c_0))$$

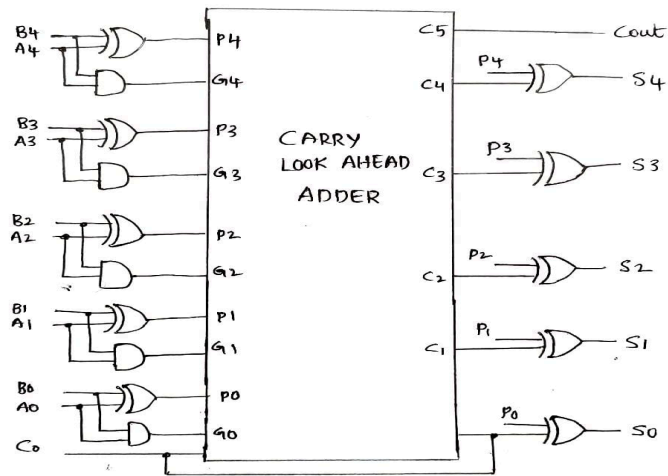
$$c_4 = g_3 + p_3(g_2 + p_2(g_1 + p_1(g_0 + p_0 c_0)))$$

$$c_5 = g_4 + p_4(g_3 + p_3(g_2 + p_2(g_1 + p_1(g_0 + p_0 c_0))))$$

Sum Block

The sum for each bit position is calculated using the XOR operation:

$$s_i = p_i \oplus c_i$$

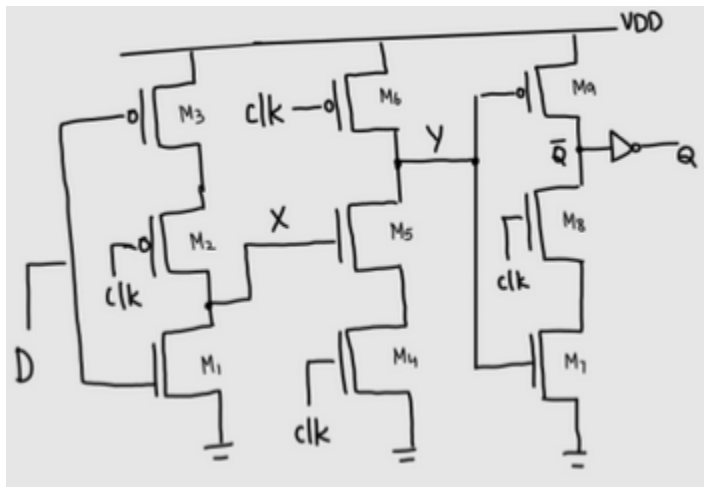


2.Design Details(Topology and Sizing)

D-Flip-Flop Register:

- Topology: 9-Transistor TSPC D-Flip-Flop.
- This topology is split into three stages: a dynamic input stage, a latching middle stage, and a static output stage. It triggers on the positive edge of the clock. By using only N-C-P (NMOS-Clock-PMOS) stacks, it avoids the need for local clock inversion and reduce skew errors.

TSPC Implementation:

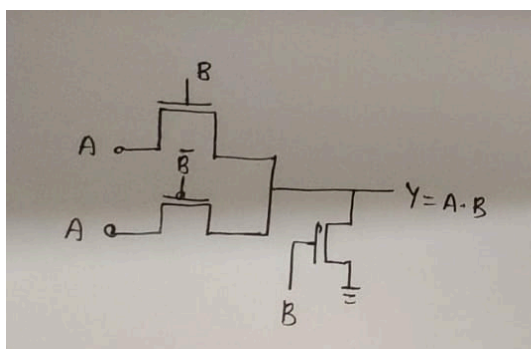


Logic Gates for Propagate & Generate:

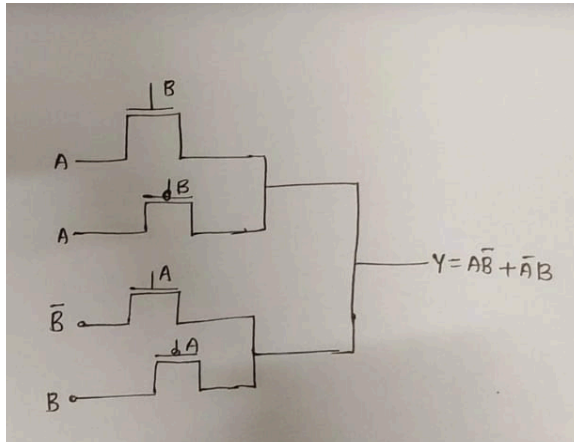
- Topology: Pass Transistor Logic (PTL).
 - XOR Gate: Implemented using a transmission gate structure $AB' + A'B$. This is used for the Propagate signal P_i .
 - AND Gate: Implemented using a modified pass-logic structure with a weak PMOS pull-up to restore the voltage level to full VDD . Since NMOS passes a weak '1'. This is used for the Generate signal G_i .

PTL implementations:

AND:

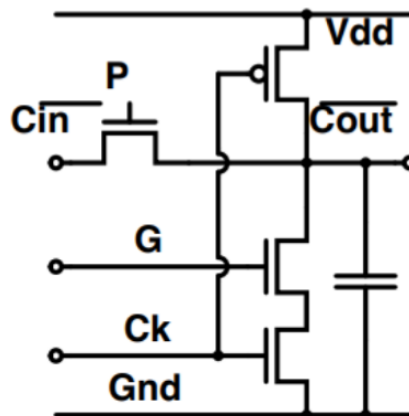


XOR:

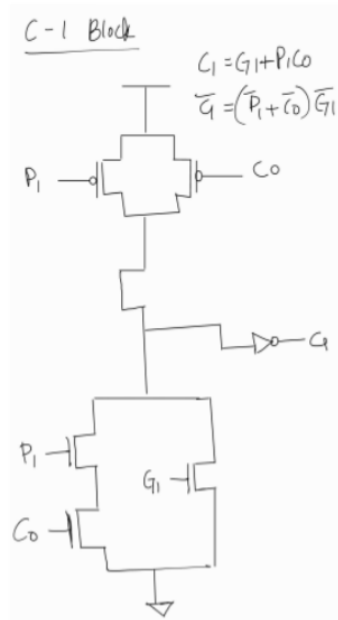


Dynamic Carry Chain of 5-bit Adder:

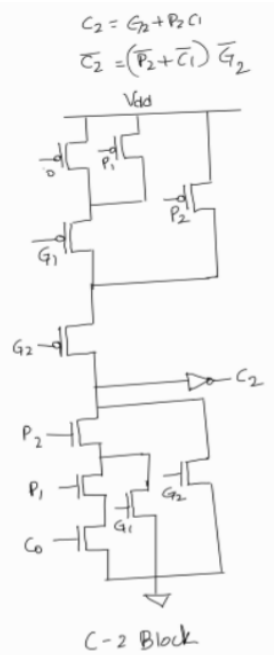
- Topology: Dynamic Manchester Carry Chain.
- This block uses a "Precharge-Evaluate" logic style.
 - Precharge: A PMOS header transistor pulls the carry node to VDD when $CLK = 0$.
 - Evaluate: A stack of series NMOS transistors discharges the node to Ground if the carry condition is met when $CLK = 1$.
 - Keeper: A weak static PMOS keeper is added to the carry node to fight leakage current and maintain the High voltage level during the evaluation phase if the node is not supposed to discharge.



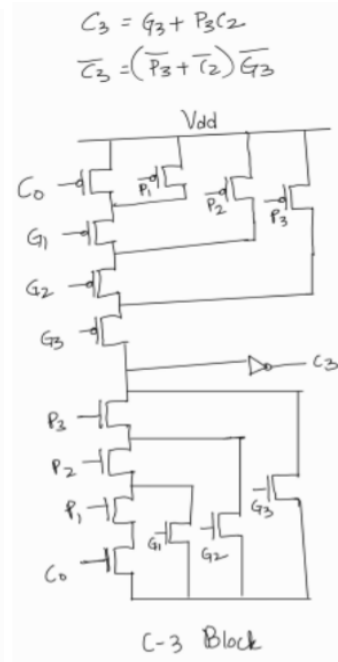
C1 :



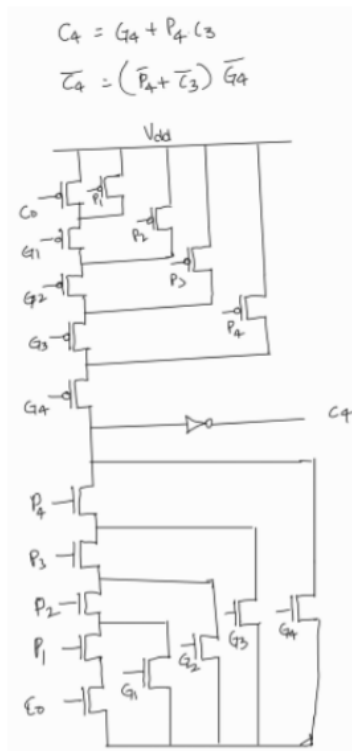
C2:



C3:



C4:

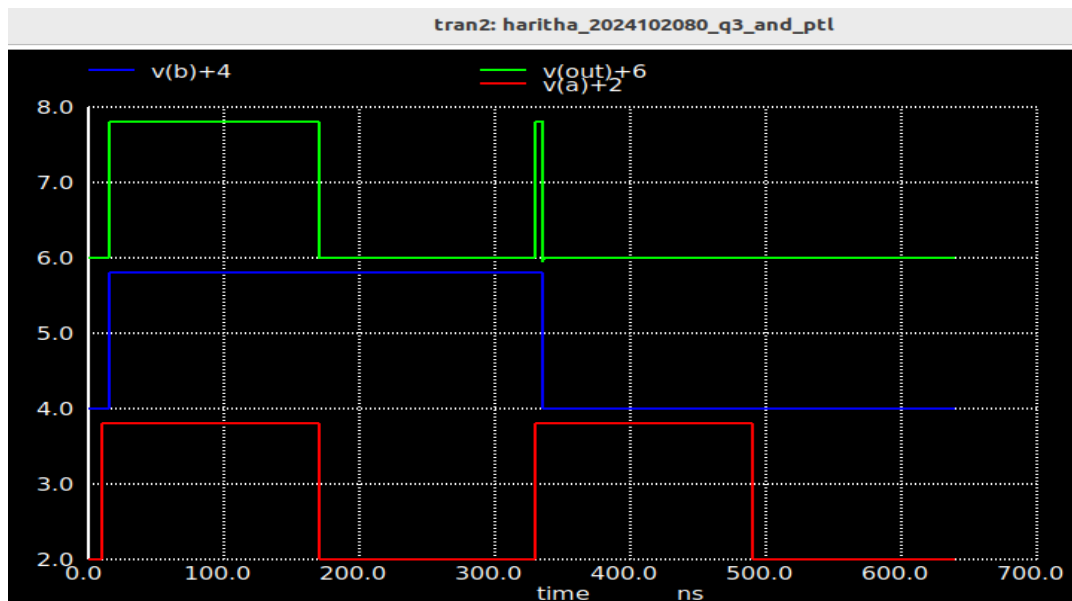


Sizing:

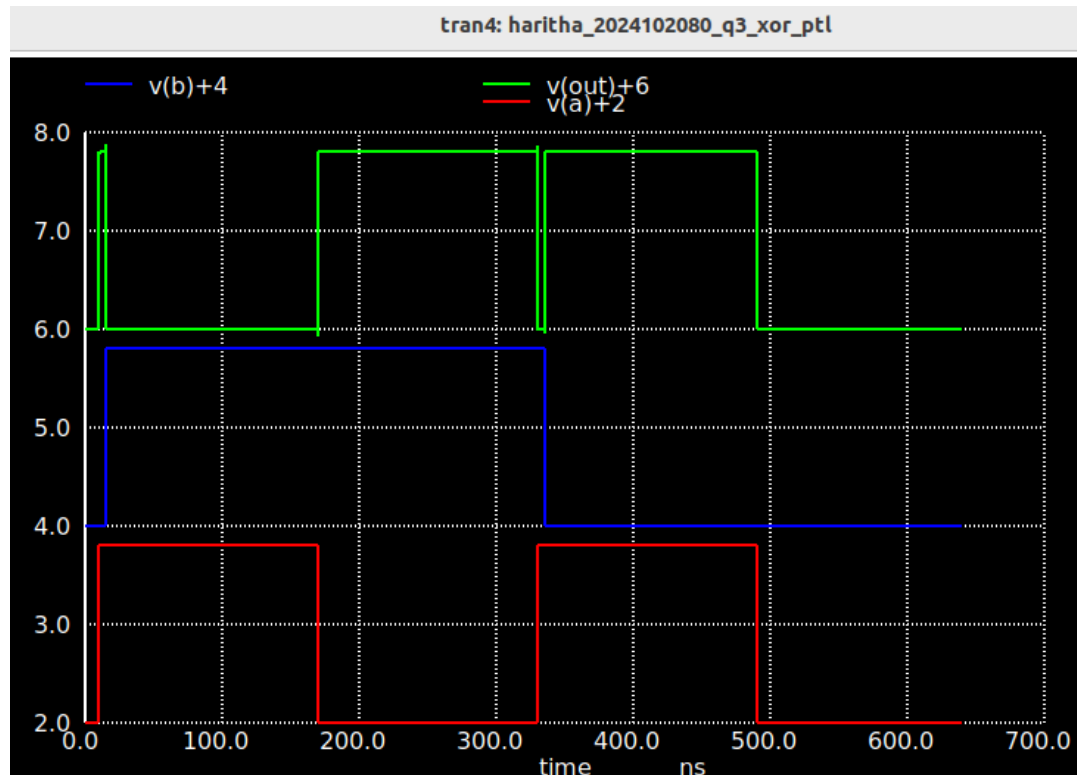
Block	Component	Width	Length
TSPC FF	PMOS/NMOS	1.8um/0.9um	180nm
P/G(PTL)	Pass Transistors	0.9um	180nm
CLA chain	Discharge NMOS	3.6 um	180nm
Load	Inverter Output	1.8um/0.9um	180nm

3. NGSPICE Simulations:

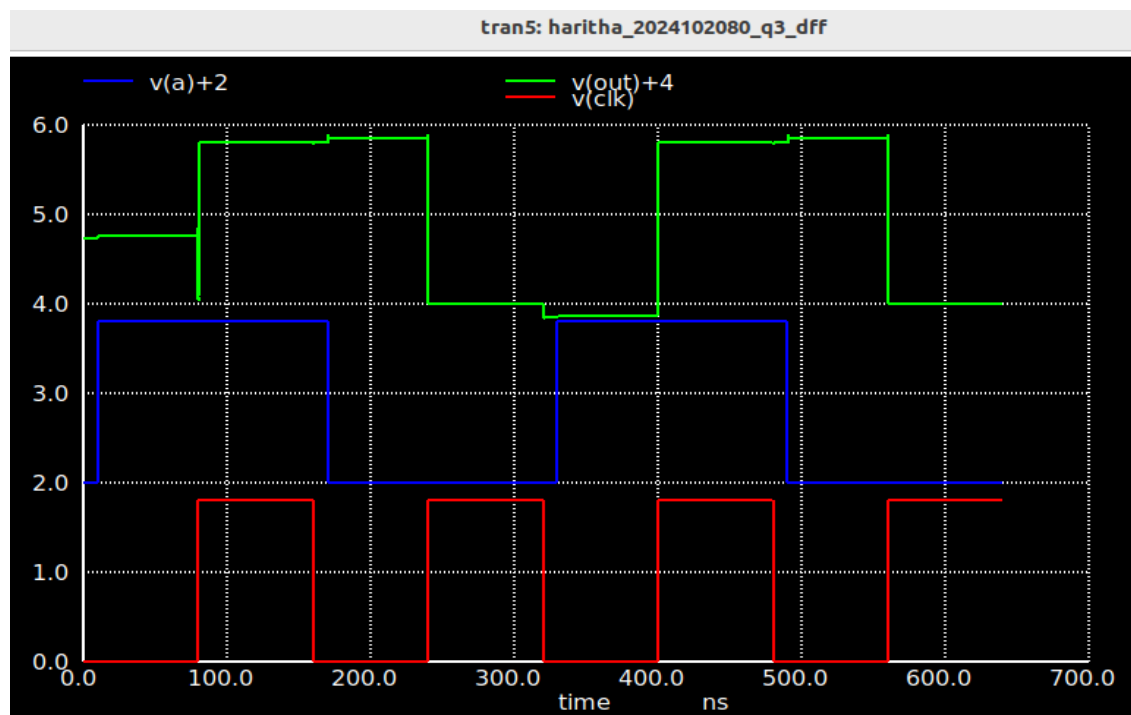
AND GATE simulation:



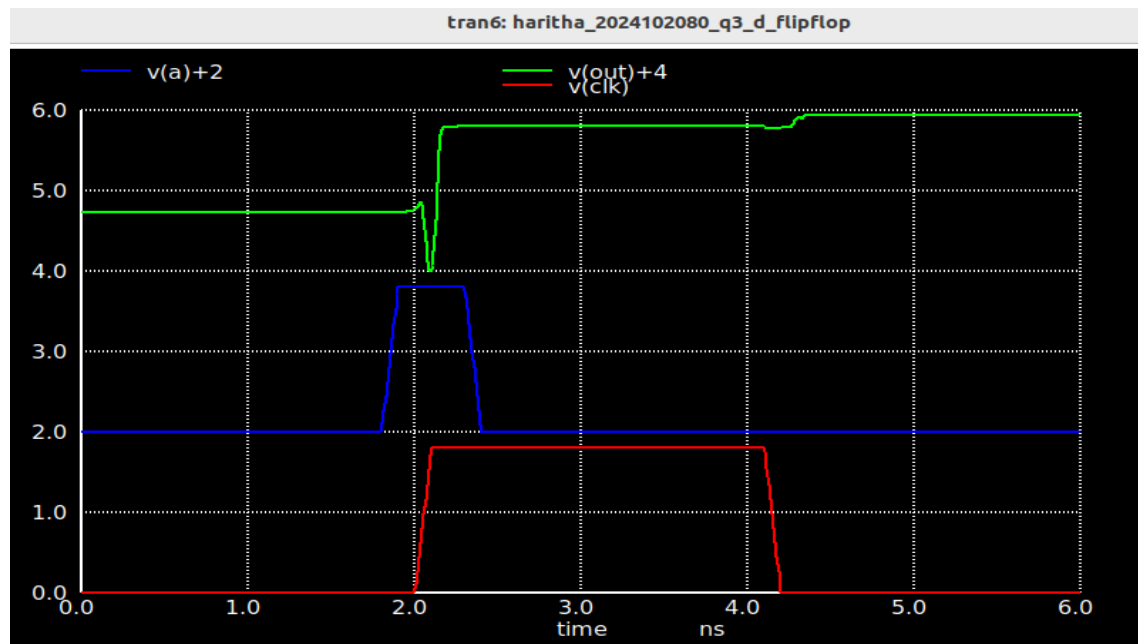
XOR GATE simulation:



D-FLIPFLOP simulation:



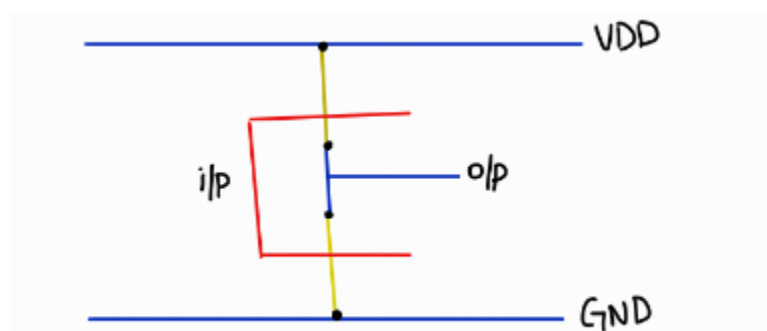
4. Setup time, Hold time, Clock to Q delay Measurements of D-flipflop:



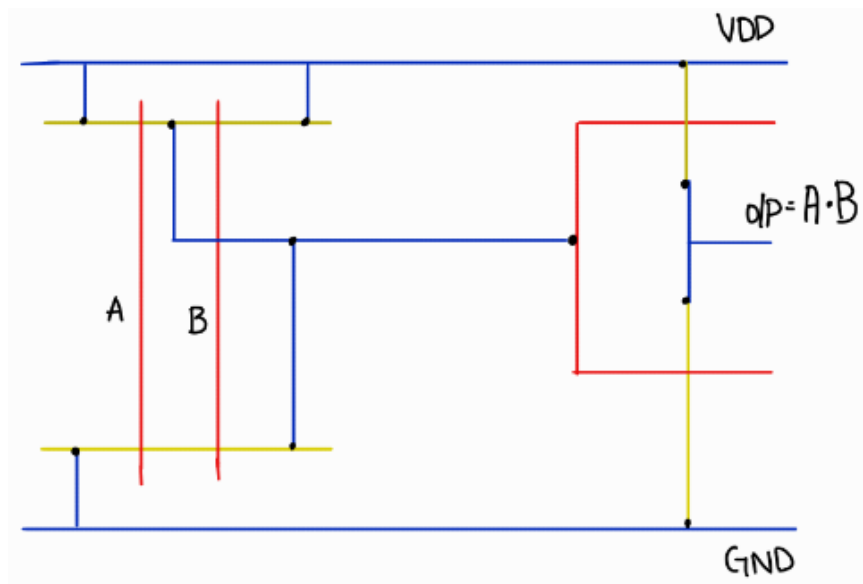
t_clk_q	=	8.064623e-11	targ=	2.130646e-09	trig=	2.050000e-09
t_setup	=	2.000000e-10	targ=	2.050000e-09	trig=	1.850000e-09
t_hold	=	3.000000e-10	targ=	2.350000e-09	trig=	2.050000e-09

5. Stick Diagrams:

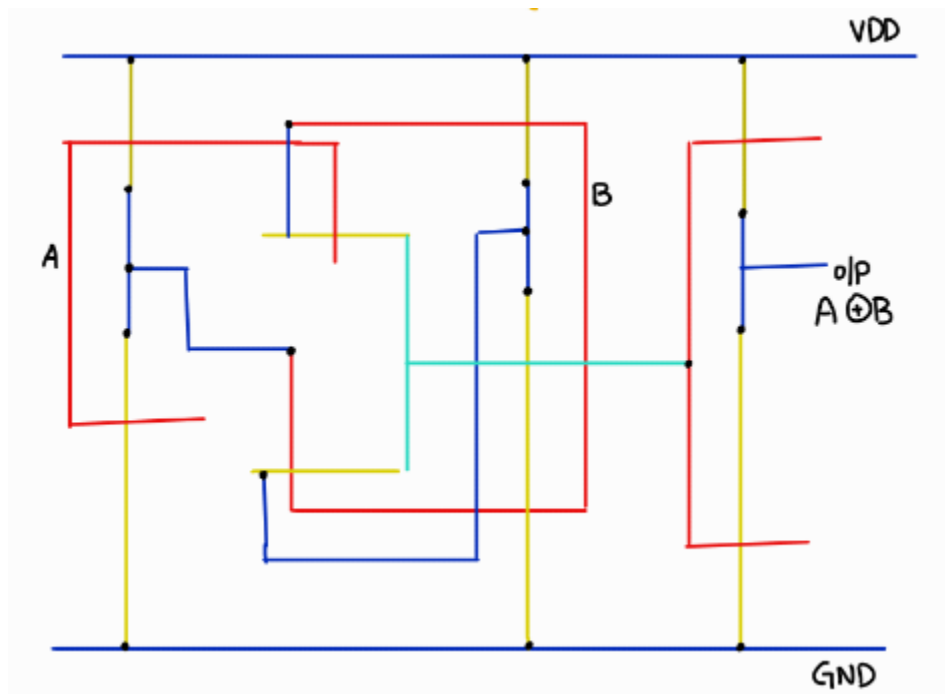
INVERTER:



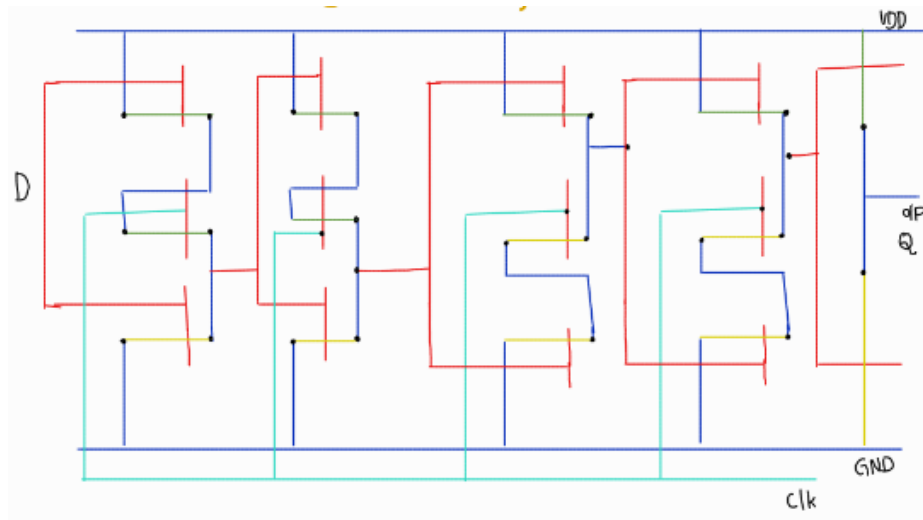
AND Gate:



XOR Gate:

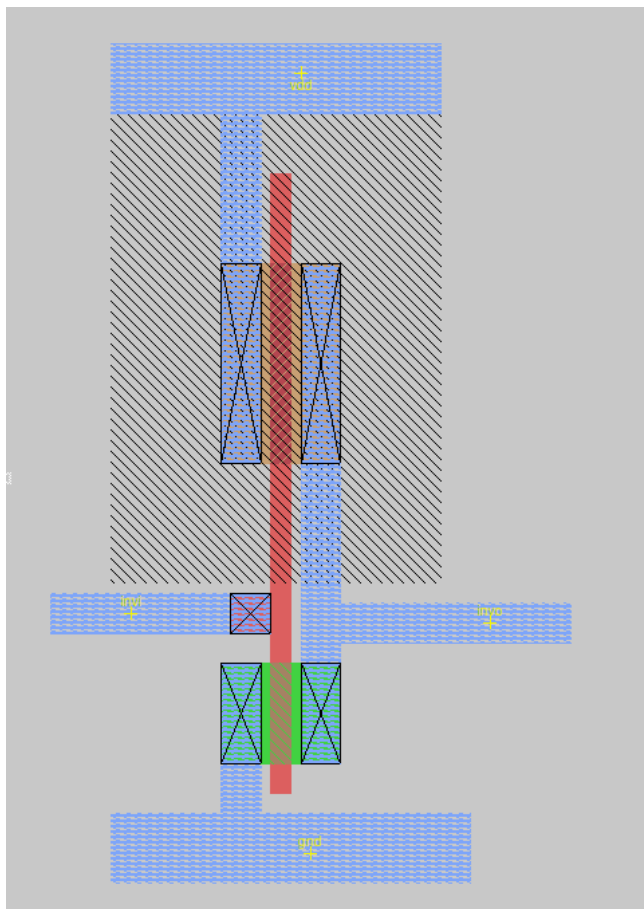


D-Flipflop:

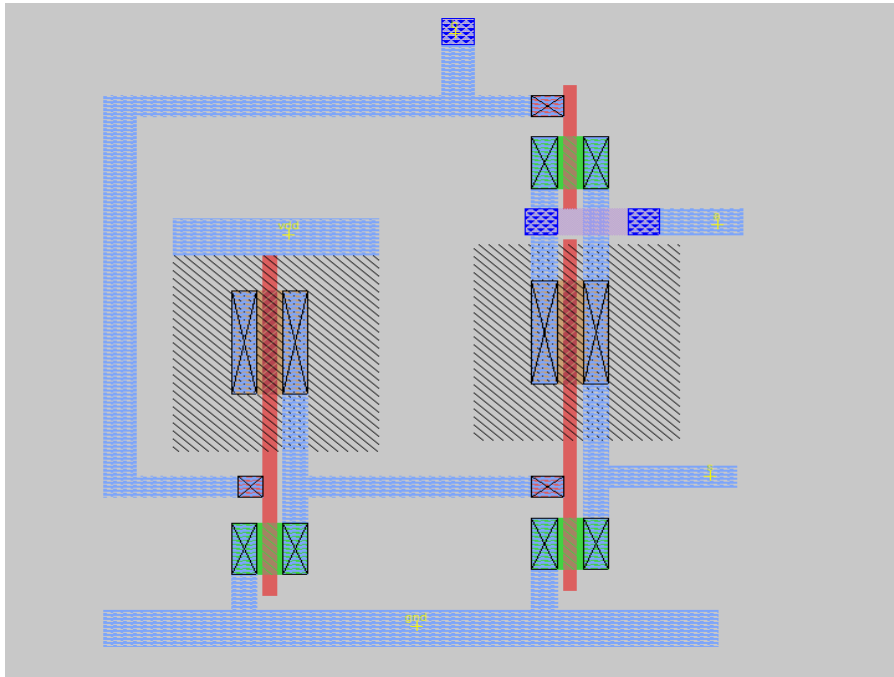


6. MAGIC Layouts of each block:

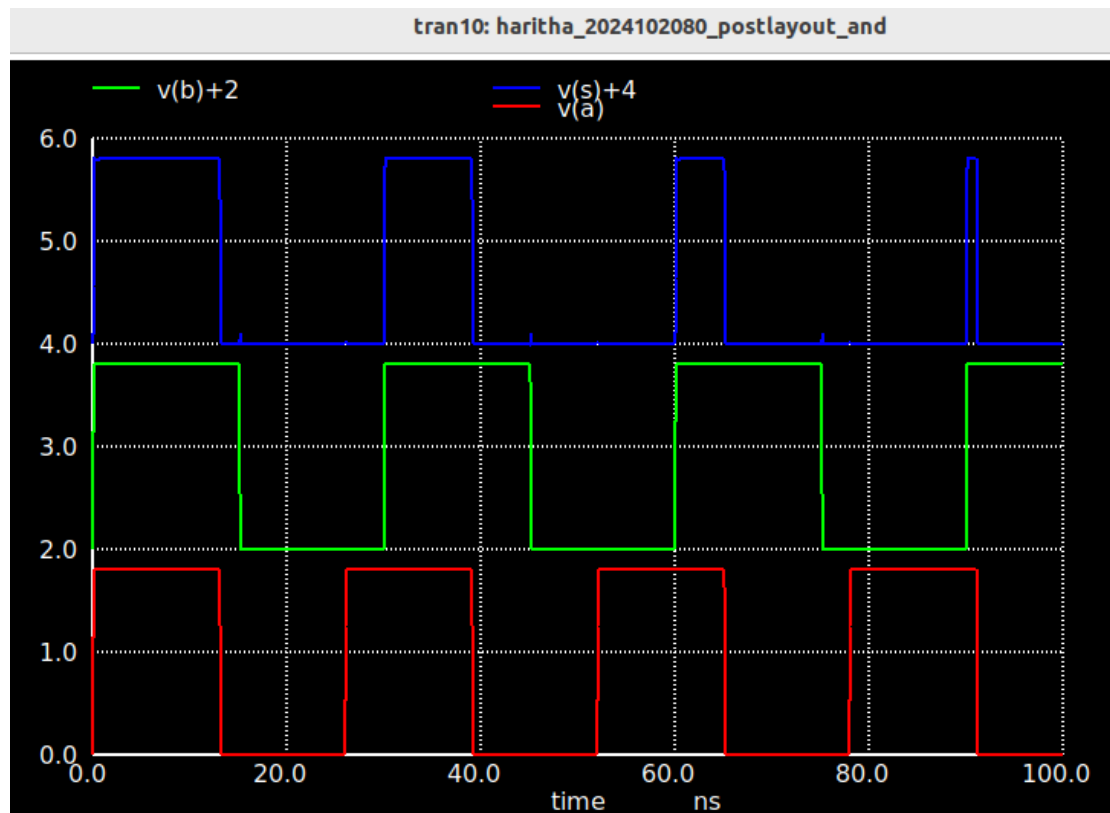
Inverter:



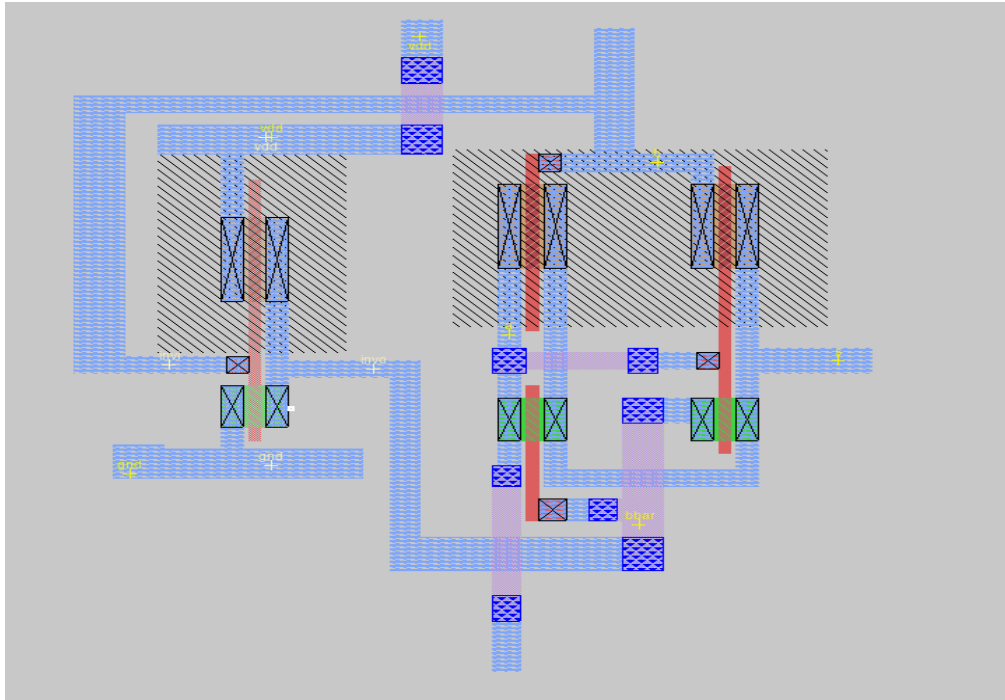
AND(Pass Transistor Logic):



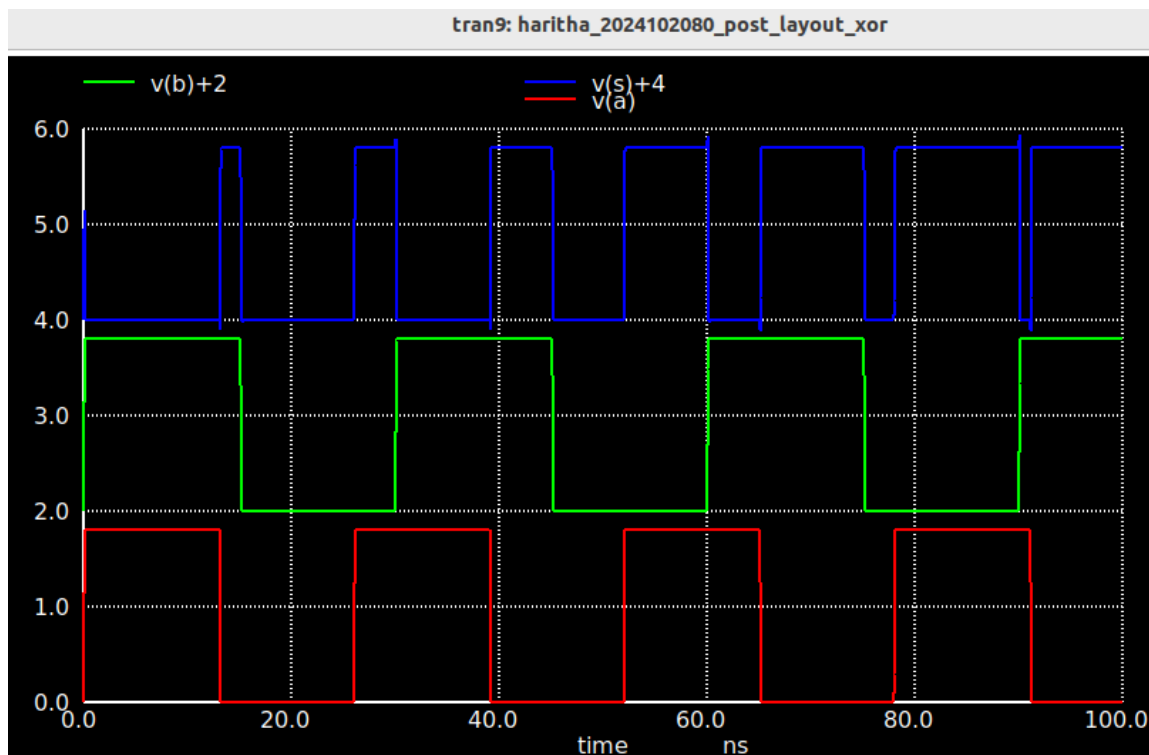
Simulation:



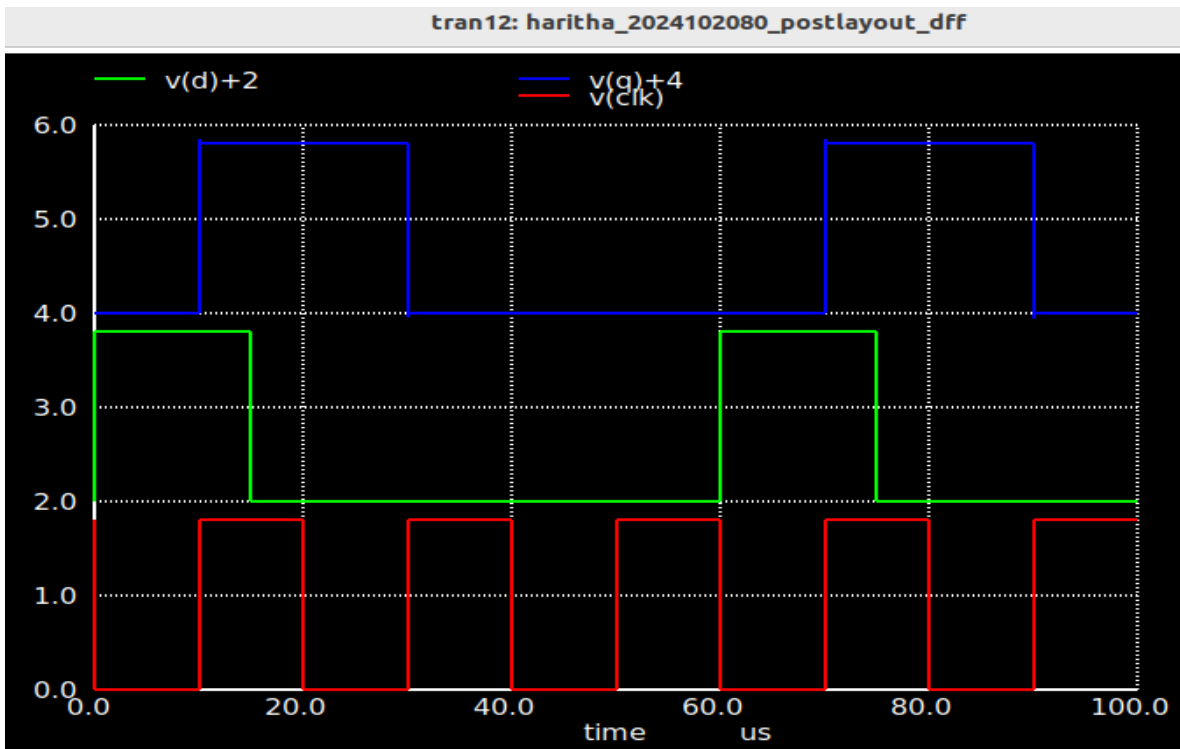
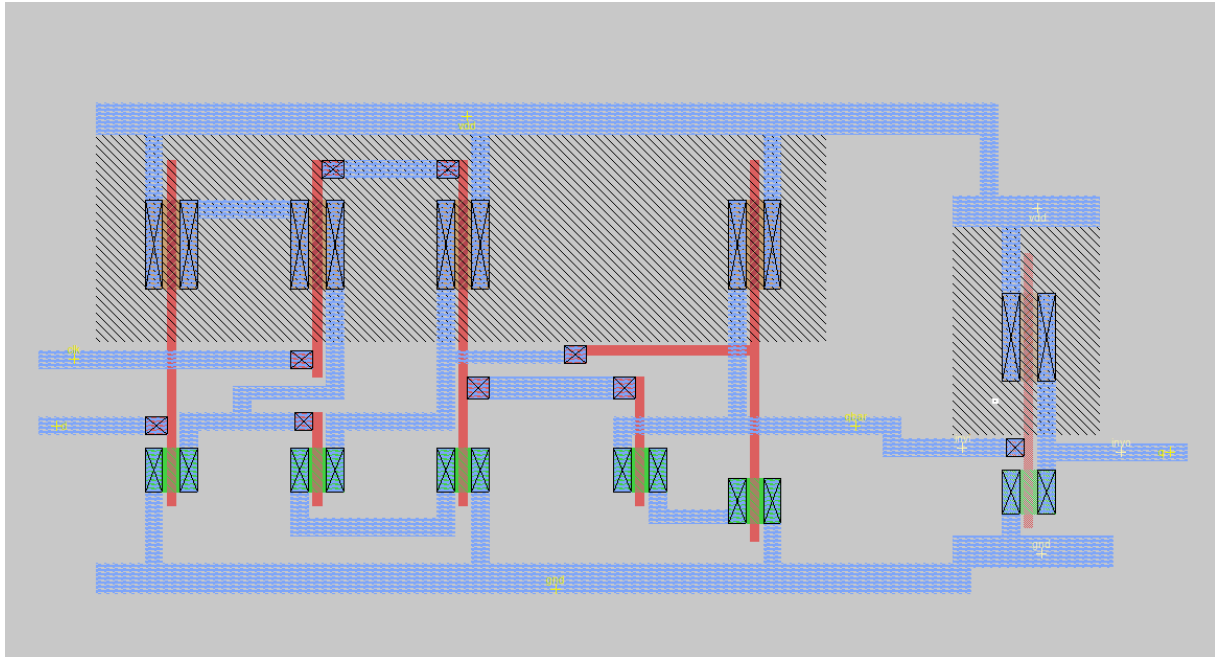
XOR(Pass Transistor Logic):



Simulation:



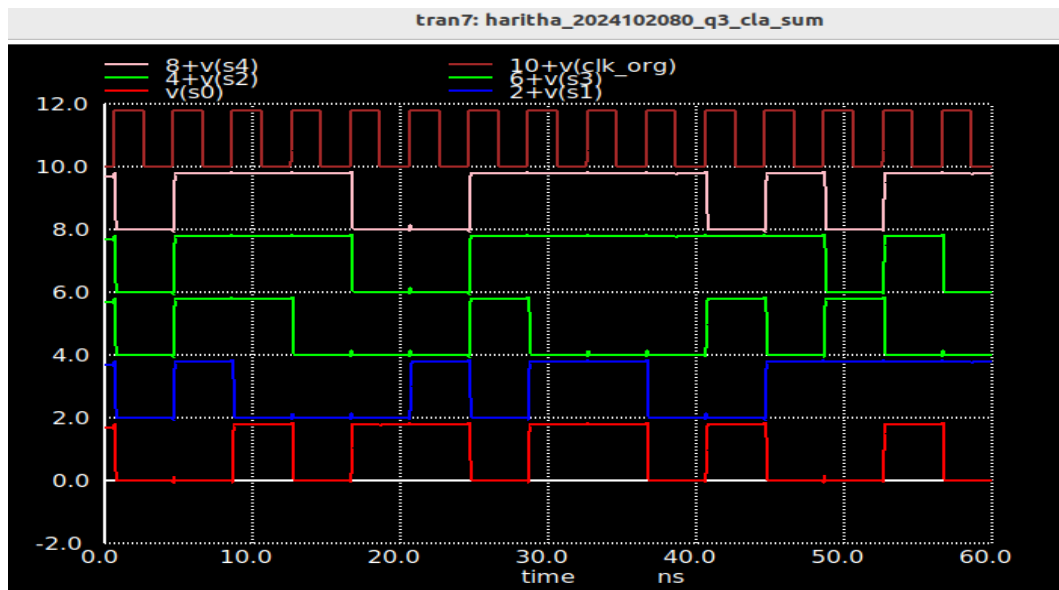
D-Flipflop:



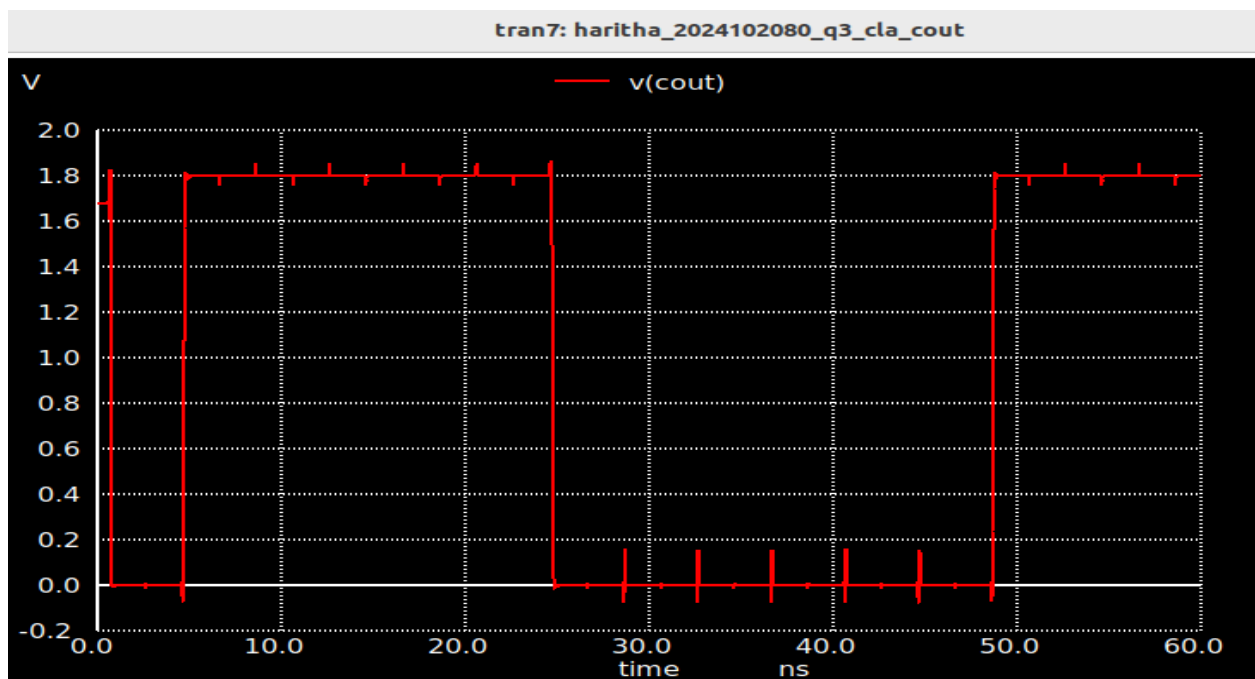
7.

5-BIT CLA simulation:

SUM:

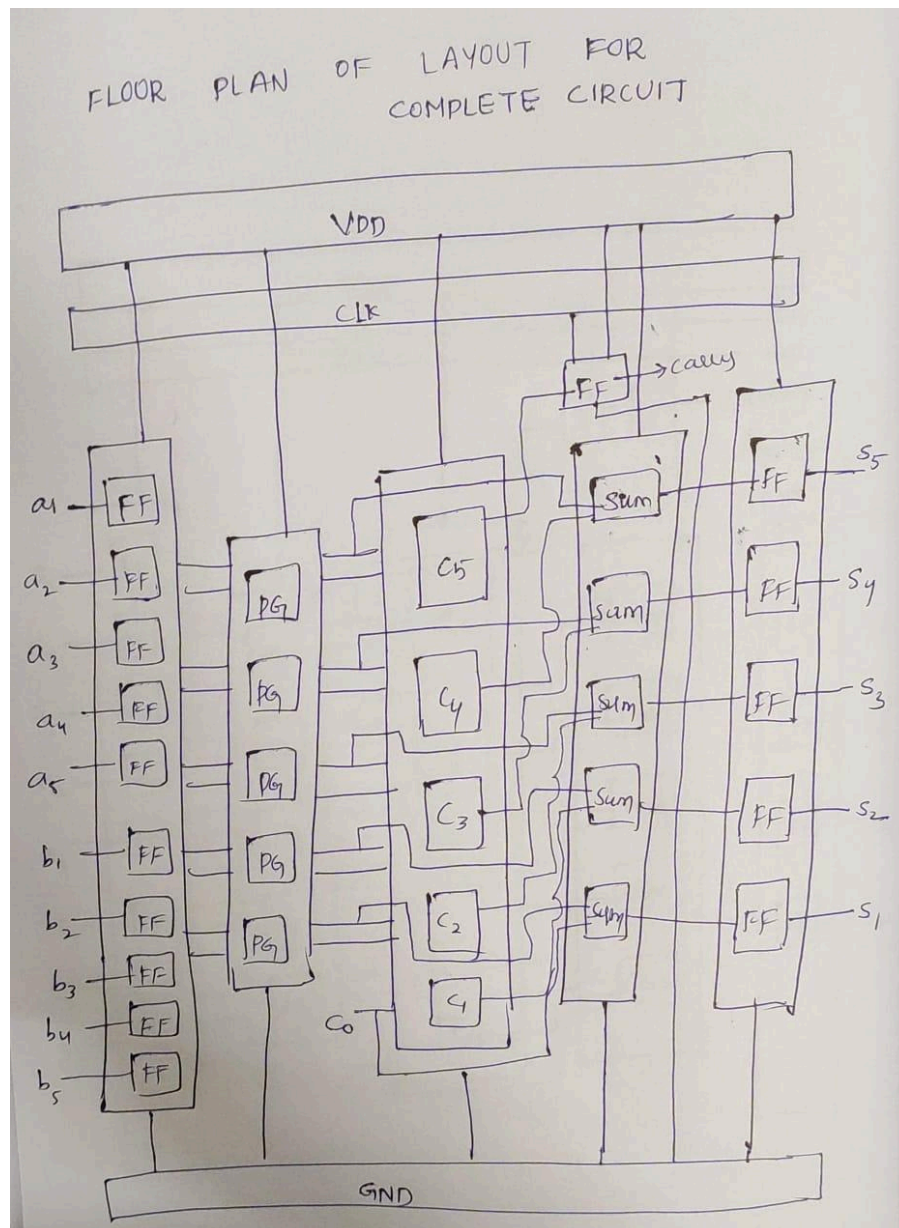


C_out:



8.

FLOOR PLAN OF THE LAYOUT



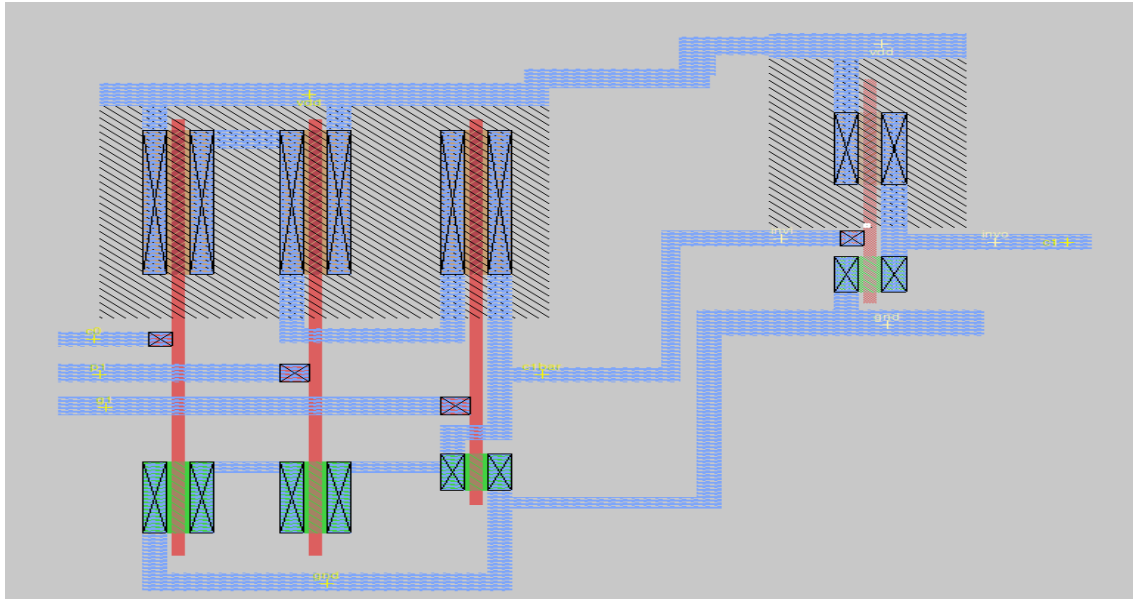
Vertical Pitch : This is defined by the height of the TSPC D-Flip-Flop cell. All other cells (XOR, AND, Dynamic Carry Chain) are designed to match this height. This constraint ensures that the Metal-1 VDD and GND power rails form continuous, unbroken lines across each horizontal bit-slice.

Horizontal Pitch: This is determined by the poly-to-poly spacing of the signal inputs. The input pins of the arithmetic core are spaced to align vertically with the output pins of the register bank, minimizing Metal-2 usage for inter-stage routing.

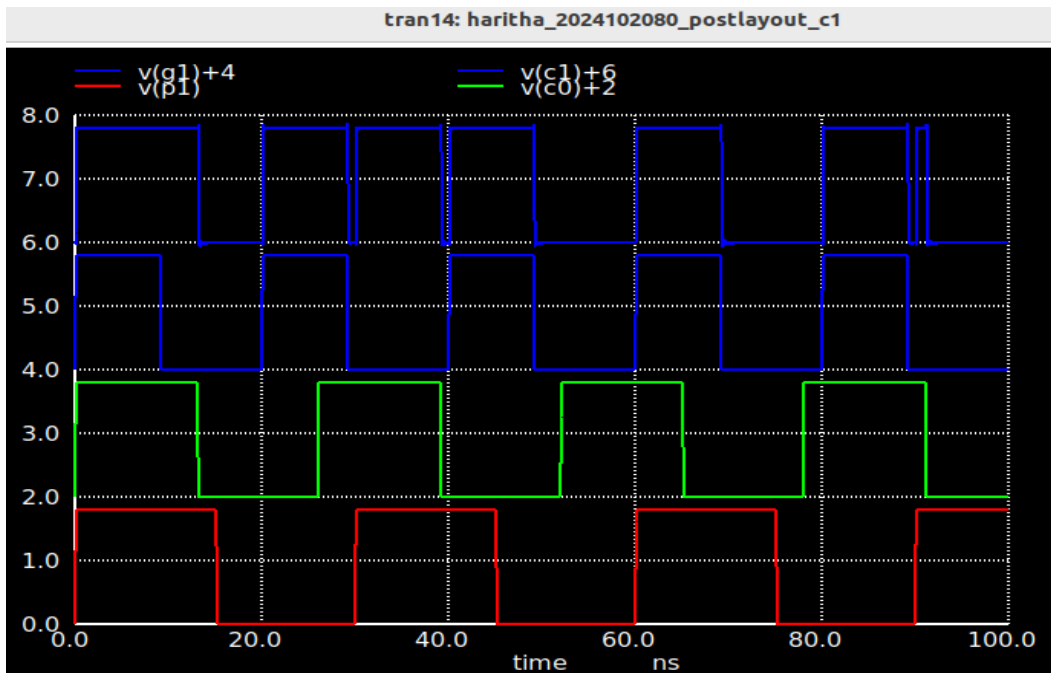
9.

Carry Blocks and Simulations:

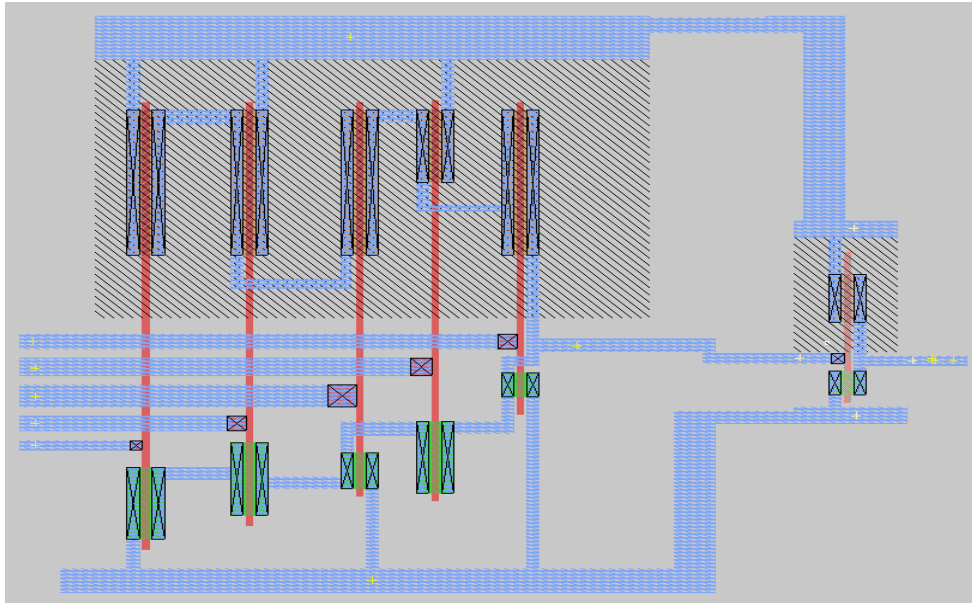
C1:



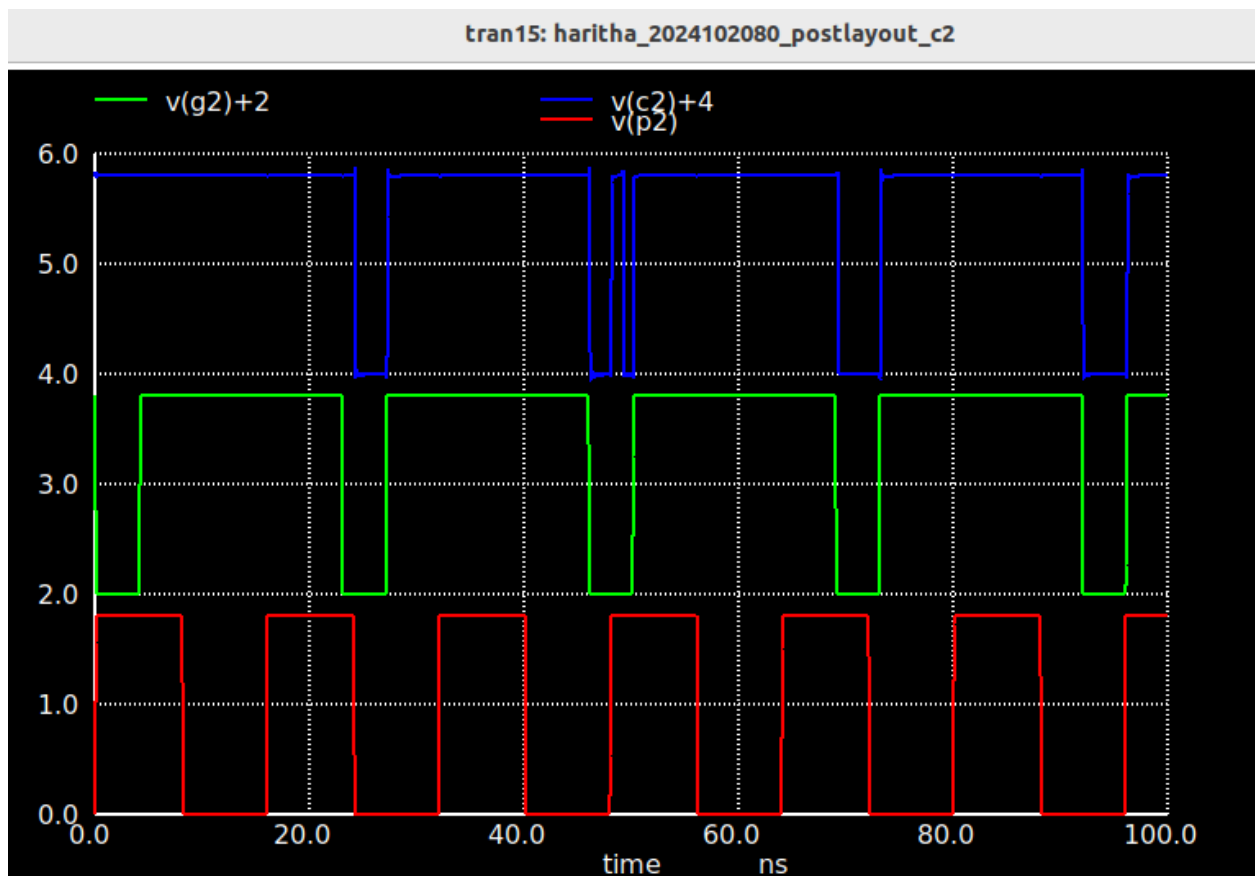
simulation:



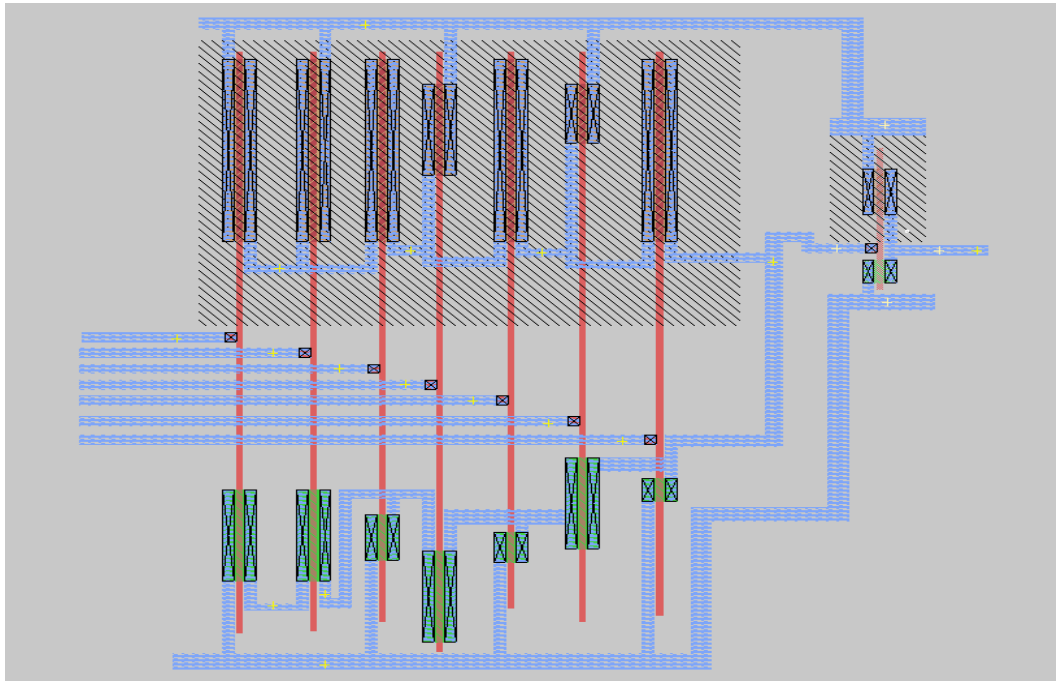
C2:



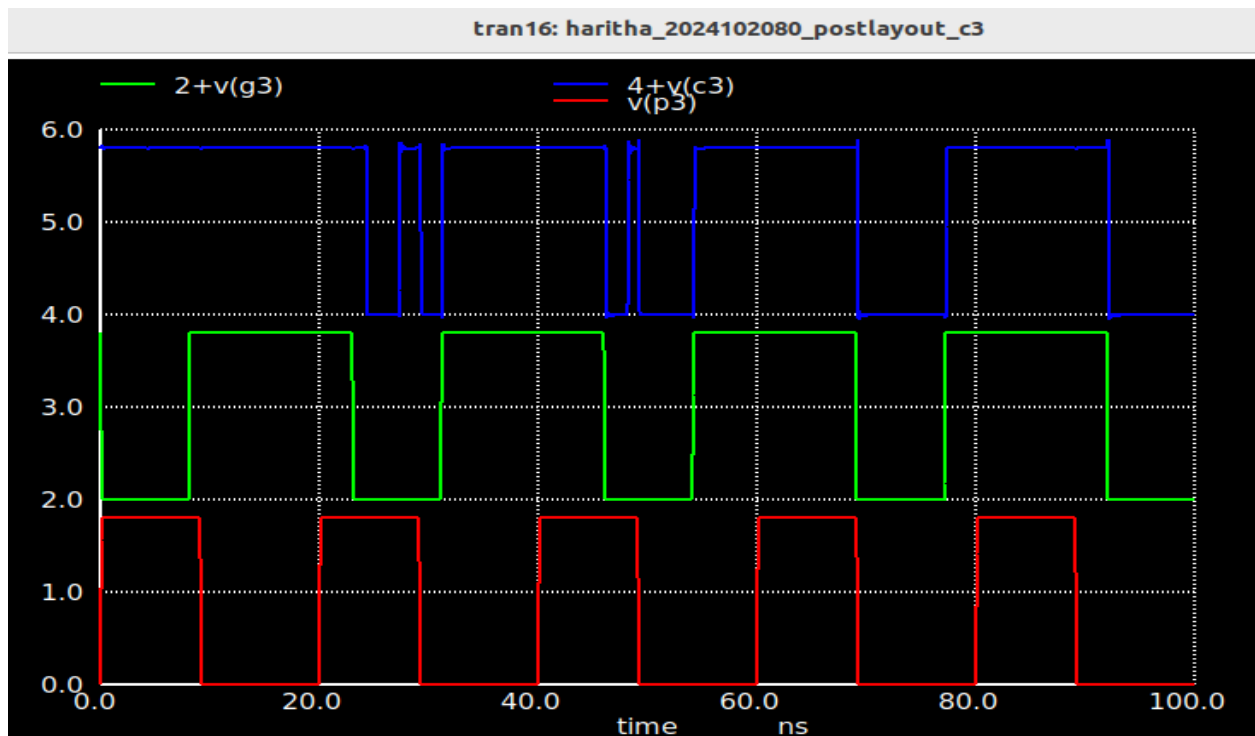
simulation:



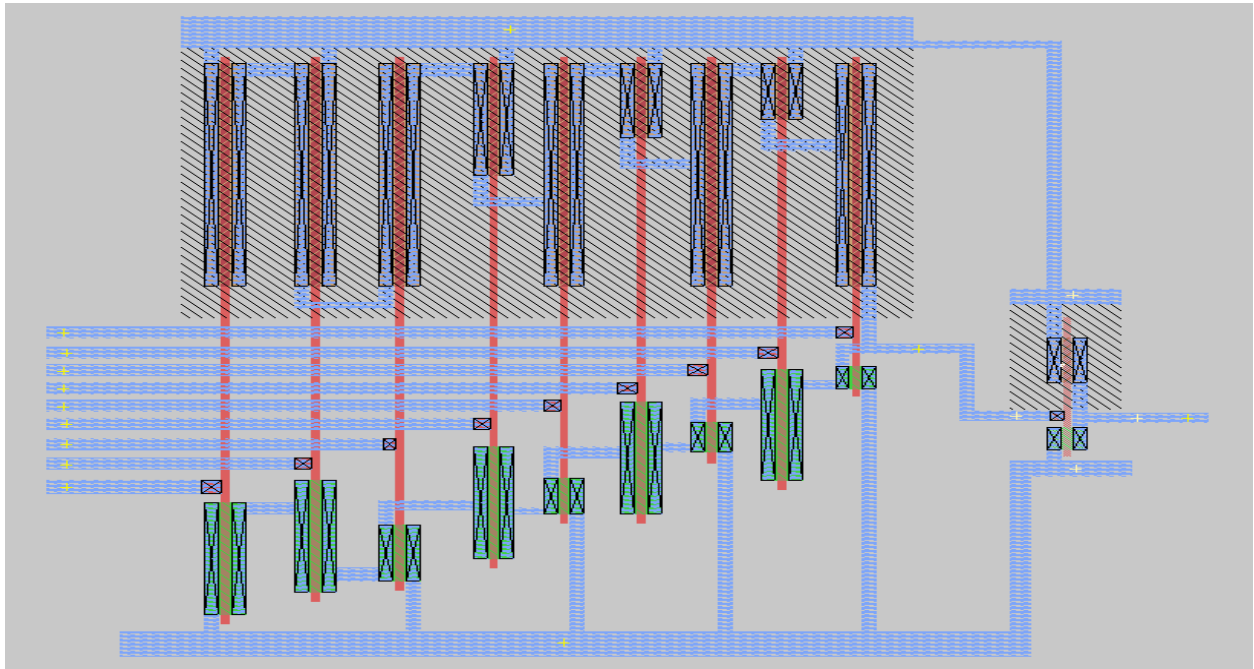
C3:



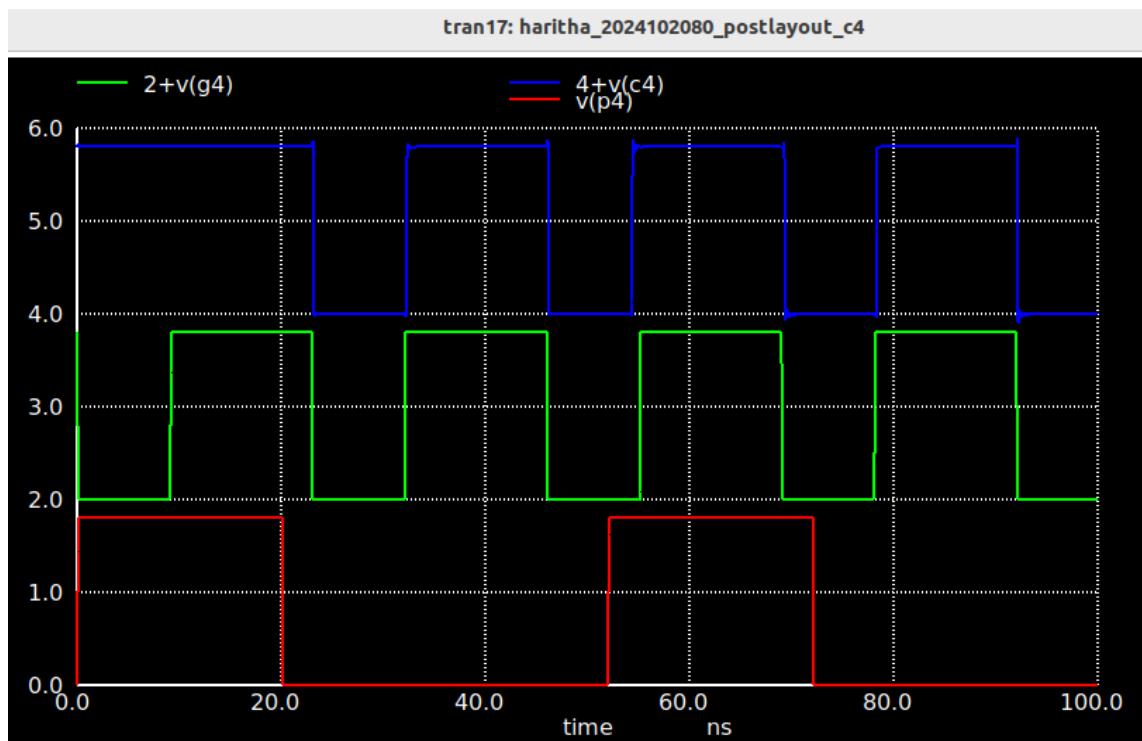
Simulation



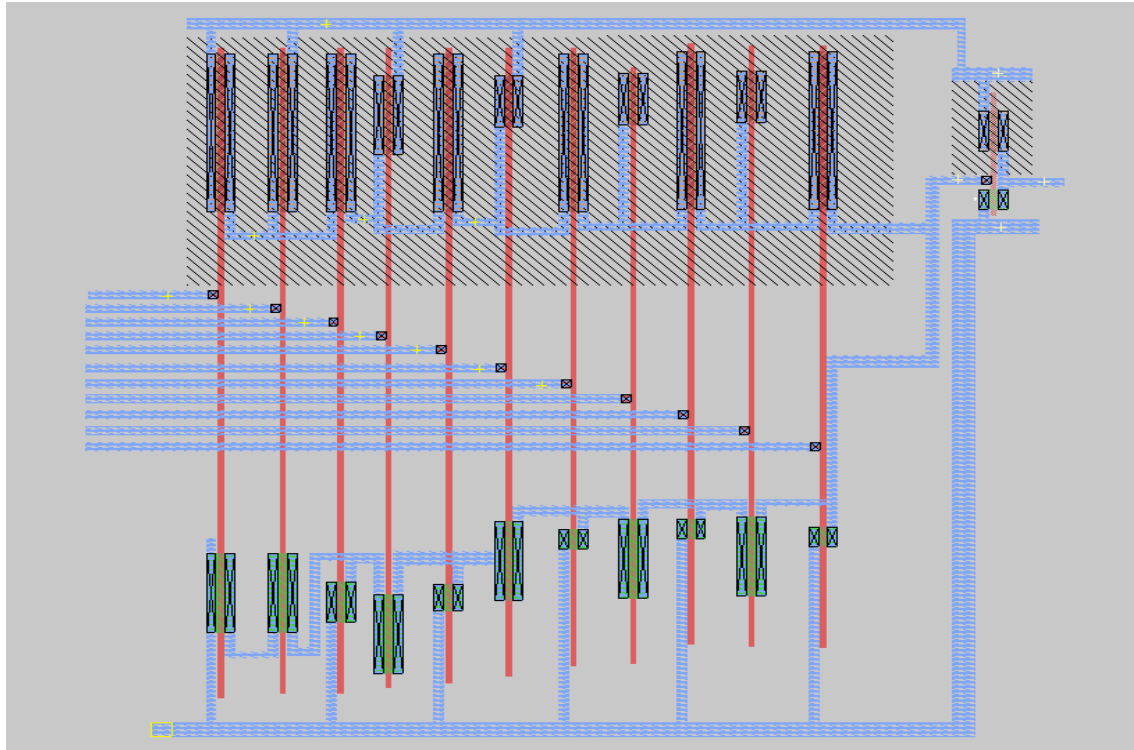
C4:



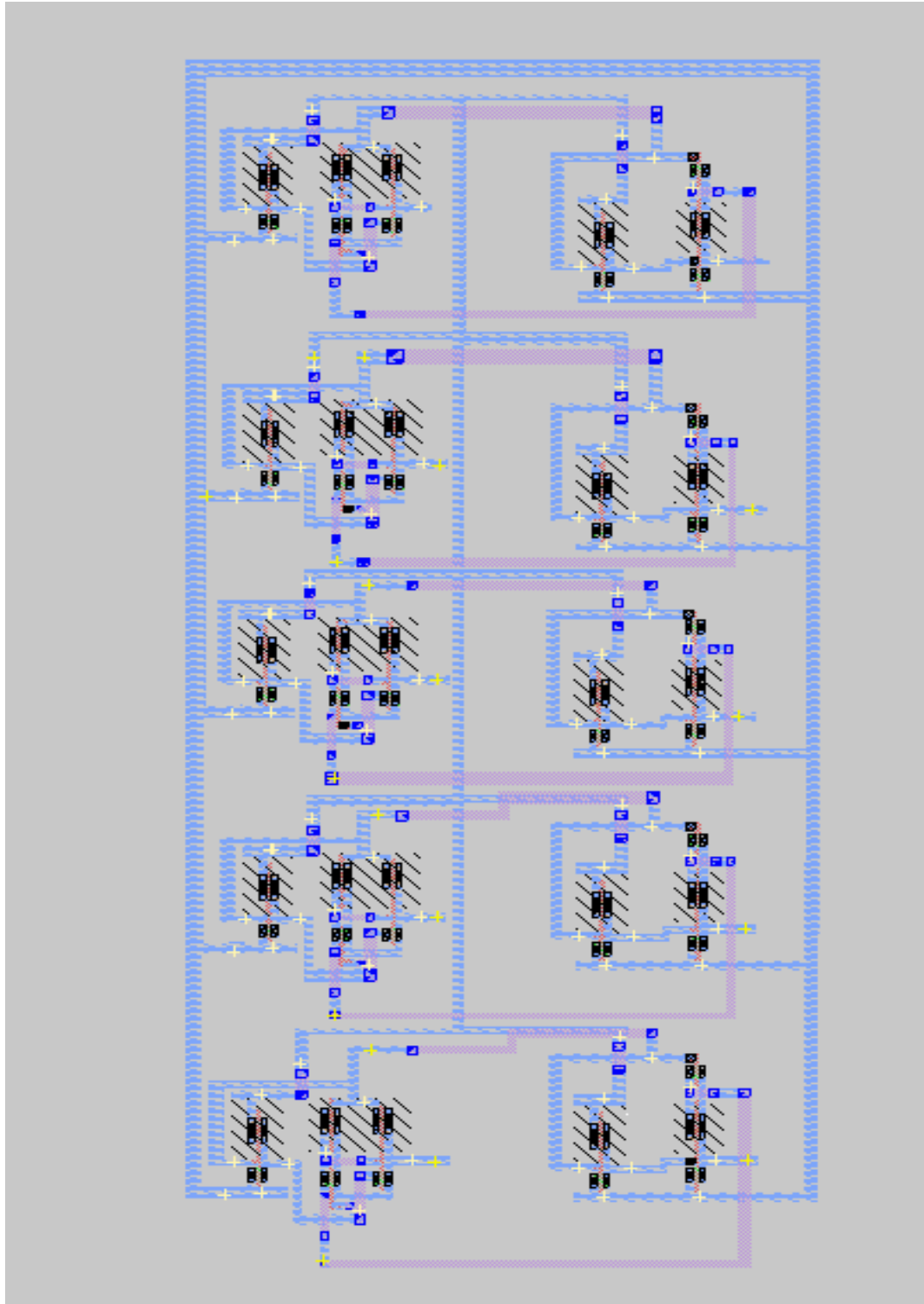
Simulation



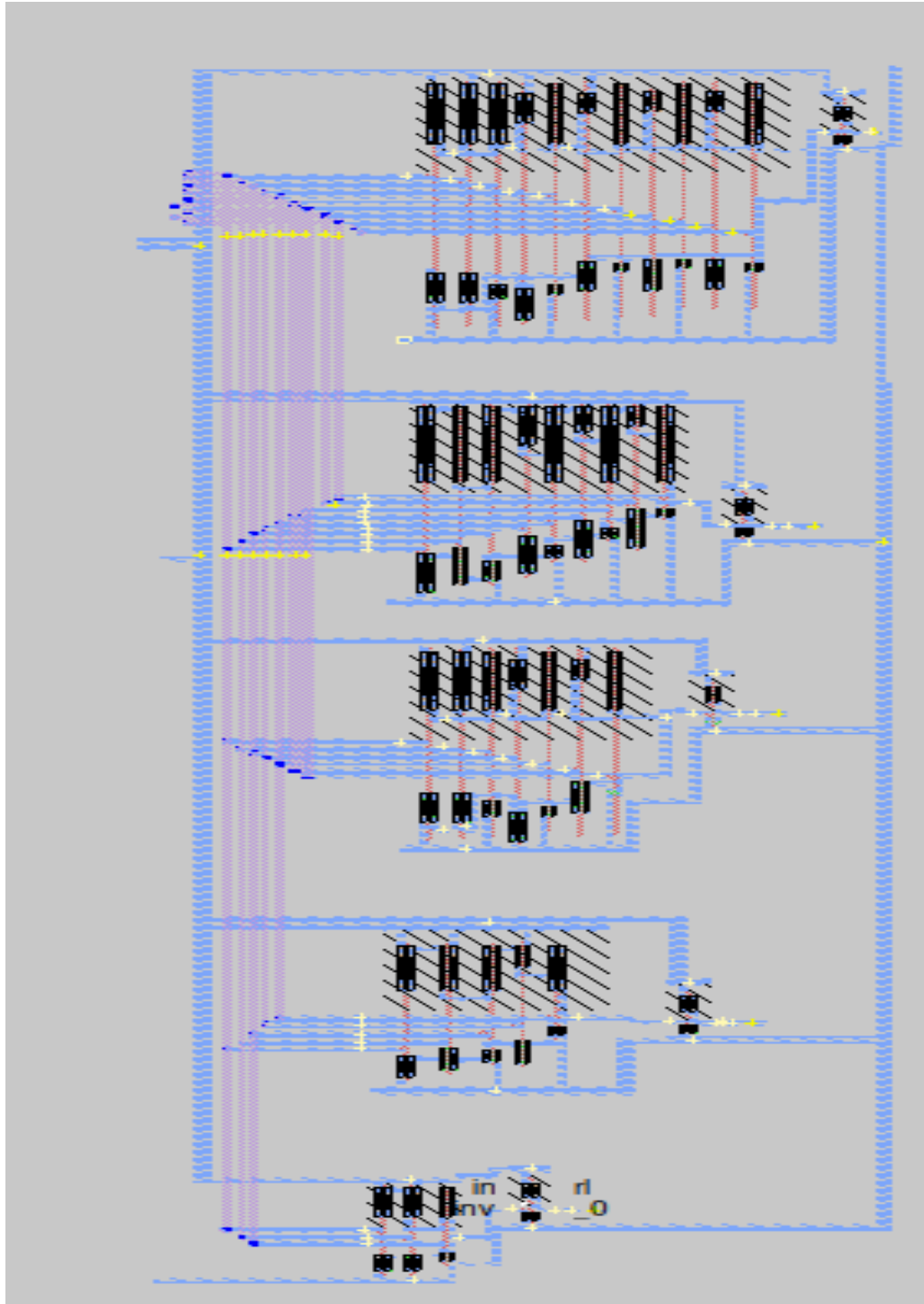
C5:



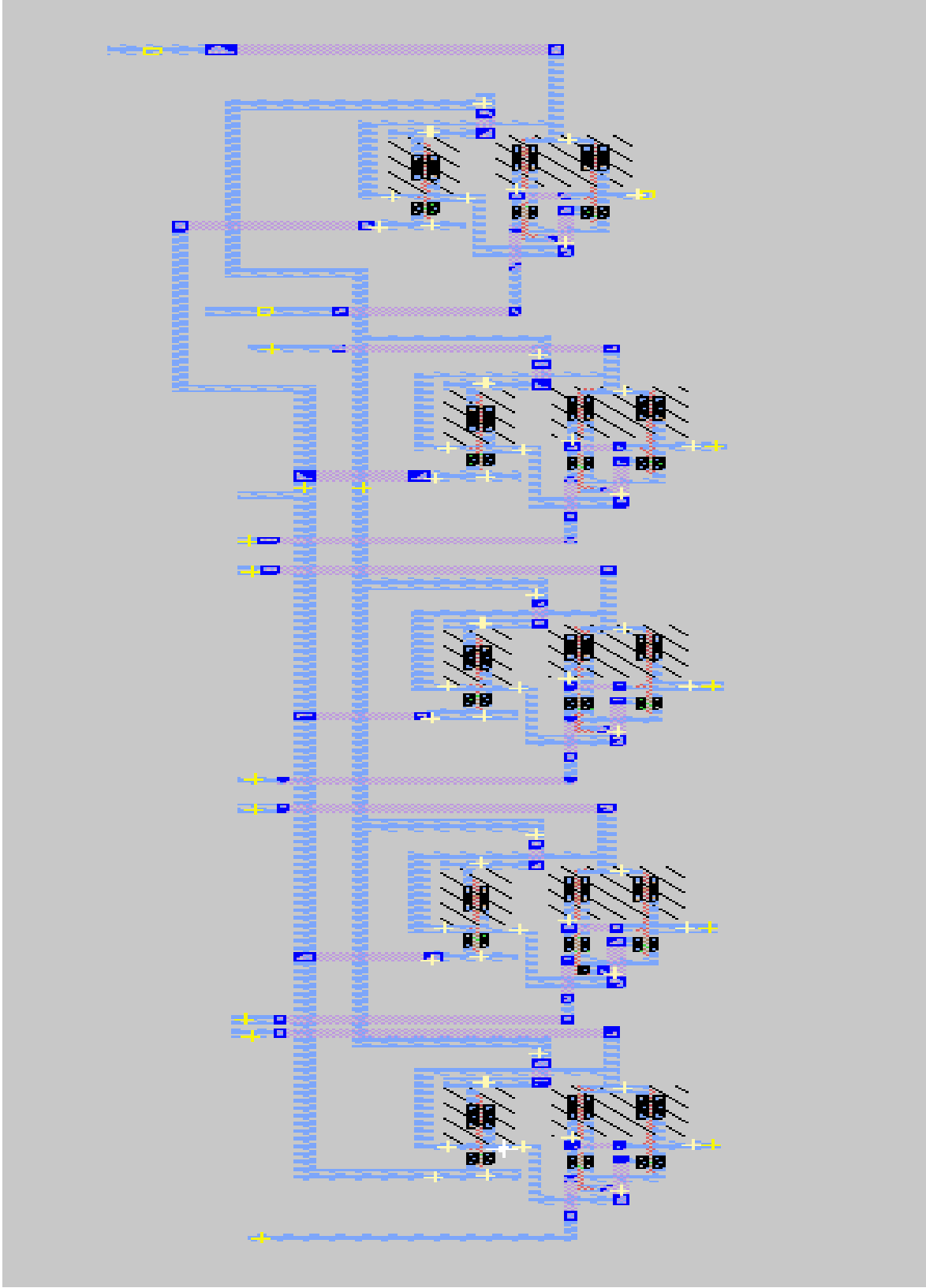
PGG Block:



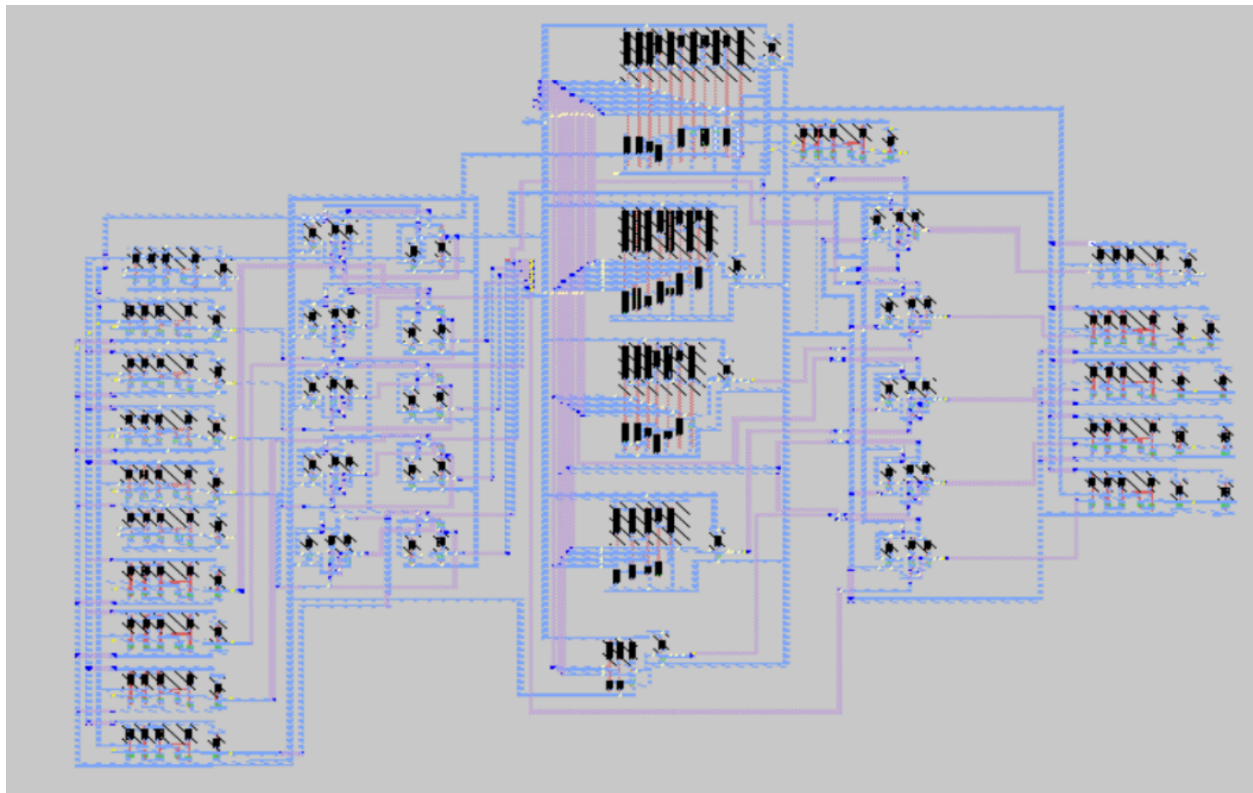
CLA Block:



SUM Block:



COMPLETE LAYOUT:



10.

In a 5-bit pipelined Carry Lookahead Adder (CLA), the critical path is the longest delay path that data travels in one clock cycle.

For this design, the signal travels through the following blocks: Flip-Flop Output (Q) → Propagate/Generate (P/G) Logic → Carry Chain (up to C5) → Sum Logic → Flip-Flop Input (D). The slowest and most delay-heavy part is the dynamic carry chain, especially when computing C4 or C5.

This is because the carry chain contains the maximum number of NMOS transistors connected in series (5 NMOS), causing the output to take longer to settle.

MEASURE DELAY and FREQUENCY-

1. Measure Propagation Delay (T_{pd}), This measures the time from the Clock Edge (trigger) to the time the MSB Carry-Out (cout) becomes valid (target). We look for the transition where cout goes HIGH or LOW.

2. Measure Sum Delay

Just in case Sum is slower than Carry (rare in CLA, but possible)

3. Calculate Max Clock Frequency

Formula: $T_{min} = t_{setup} + t_{clk-q} + t_{logic_delay}$

Since we are measuring from Clock → Output, our measurement captures ($t_{clk-q} + t_{logic}$).

We just need to add the Setup Time (0.2ns from your previous finding).

$t_{pd_cout} = 0.07ns$.

$T_{min} = 0.07ns + 0.20ns \text{ (Setup)} = 0.27ns$

$F_{max} = 1 / T_{min} = 3703Mhz$

```
t_pd_cout      = 7.938691e-11 targ= 4.684387e-09 trig= 4.605000e-09
t_pd_sum4      = 7.938451e-11 targ= 4.684385e-09 trig= 4.605000e-09
```

11.

Verilog simulations:

Time	Cin	A	B	Sum	Cout
0	0	00001	00000	00001	0
25000	0	00011	00010	00101	0
55000	0	00101	00100	01001	0
85000	0	01001	01000	10001	0
115000	0	10001	10000	00001	1
145000	1	01110	10101	00100	1
175000	0	00011	11100	11111	0
205000	1	11000	00111	00000	1

GTK waveform:

